WEEK 1: DYNAMIC PROGRAMMING

Advanced Algorithms

ROADMAP

1. Longest Path in a DAG
2. Bellman–Ford: DP on Graphs with Cycles
3. Maximum Subarray Sum
4. Longest Increasing Subsequence
5. Coin Change

One technique, five problems — plus a couple of warm-up variations along the way.

DYNAMIC PROGRAMMING

Dynamic programming is a versatile algorithmic technique that can be used to solve a variety of problems.

The hallmark is decomposing a problem into a sequence of *subproblems*, which progressively build up to a solution of the original problem.

Rather than formulate a theoretical framework for dynamic programming, we are going to illustrate it by means of examples.

LONGEST PATH IN A DAG

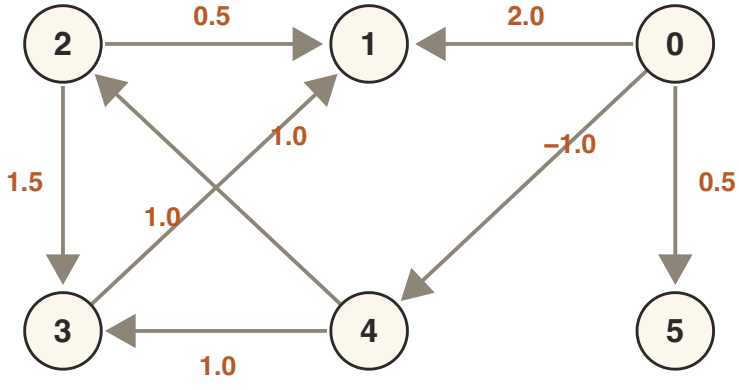
LONGEST PATH IN A DAG

The problem of finding a longest path in a DAG is a canonical dynamic programming problem.

Many other (seemingly unrelated) problems can be cast in this framework.

> **PLAN** Solve *shortest* paths in a DAG first — the longest path then comes for free.

SHORTEST PATHS IN A DAG



A DAG with edge weights, source vertex 0.

Perhaps you recall the single-source shortest path problem in a DAG.

We can allow negative edge weights: there will be no negative-weight cycles, as there are no cycles at all.

Multiplying all weights by -1, the longest path becomes the shortest path.

STEP 1: TOPOLOGICAL SORT

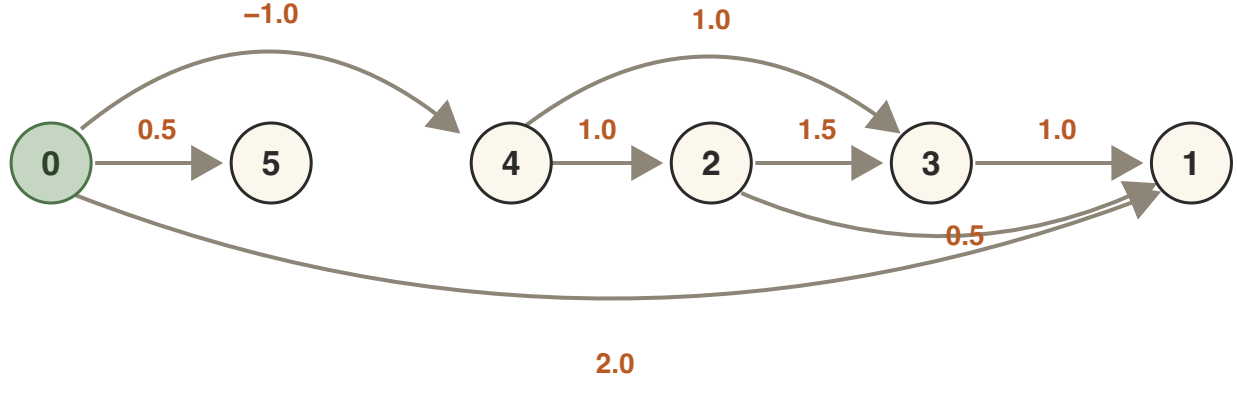
Step 1: Compute a topological sort of the graph.

> **DEFINITION** A topological sort is an ordering of the vertices such that $u < v$ for every edge (u, v).

We can compute a topological order by depth-first search or Kahn's algorithm in time $O(|V| + |E|)$ in the adjacency list model.

THE GRAPH, UNTANGLED

Laying the vertices out in topological order $0, 5, 4, 2, 3, 1$ — every edge now points to the right:



Same DAG, vertices in topological order (source in green).

RELAXING AN EDGE

To relax the edge $e = (u, v)$ we check if

$$\texttt{dist\_to}[u] + e.\texttt{weight} < \texttt{dist\_to}[v]$$

If so, then we do the update:

$$\texttt{dist\_to}[v] = \texttt{dist\_to}[u] + e.\texttt{weight}$$

Note that $\texttt{dist\_to}[v]$ never increases under edge relaxation.

STEP 2: RELAX IN TOPOLOGICAL ORDER

Step 2: Relax the outgoing edges of each vertex following the topological order.

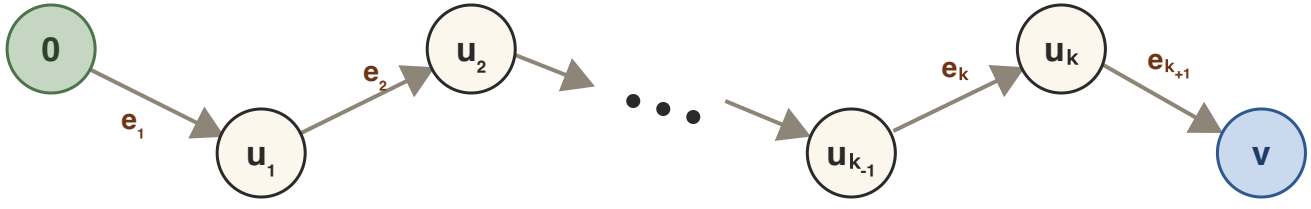
```
for (auto v : topo_order) {
    for (const auto& edge : adj_list[v]) {
        relax(edge);
    }
}
```

This takes time $O(|V| + |E|)$ as each edge is relaxed exactly once.

RESULT

Shortest path in a DAG There is an algorithm to find the shortest path in a DAG in time $O(|V| + |E|)$ in the adjacency list model.

WHY IT WORKS



If this is a shortest path from 0 to vertex v, we know that

$$0 < u_1 < u_2 < \cdots < u_k < v$$

in the topological order.

By relaxing the outgoing edges of vertices in topological order we relax e_1 before e_2 before e_3, all the way to e_{k+1} — that is, we relax this entire path in order.

WHY IT WORKS

Relax-a-path property If the algorithm relaxes a shortest path from 0 to v in order, then $\texttt{dist\_to}[v] = d(0, v)$.

Each relaxation only ever lowers `dist_to`, and after the whole path is relaxed the value has reached the true distance.

LONGEST PATH

COROLLARY There is an algorithm to find the longest path in a DAG in time $O(|V| + |E|)$.

Reason: multiplying all weights by -1 turns the longest path into the shortest path.

BELLMAN–FORD:
DP ON GRAPHS WITH CYCLES

DROPPING THE “A” IN DAG

New problem: single-source *shortest* paths in a general directed graph — cycles allowed, and edge weights may be negative.

Dijkstra’s algorithm cannot handle this because of the negative weights.

The DAG algorithm cannot handle this because a graph with cycles has *no topological order*.

WHAT'S THE ORDERING?

In the DAG algorithm, the subproblems were ordered for us: process vertices in topological order. That order is gone.

A recurring DP move: when the natural order disappears, *add a parameter* that restores one.

> **IDEA** Order subproblems not by vertex, but by the *number of edges* a solution is allowed to use.

THE SUBPROBLEM

Definition $d_k(v)$ = minimum weight of a *walk* from s to v using at most k edges (∞ if there is none).

A walk may revisit vertices and edges. We don't forbid that up front — the budget k keeps every subproblem finite, and we will see that the optimum has no reason to revisit anything.

Base case ($k = 0$): $d_0(s) = 0$ and $d_0(v) = \infty$ for $v \neq s$.

THE RECURRENCE

Consider a best walk to v using at most k edges, and look at its *last edge*.

Either the walk uses at most $k-1$ edges, or it arrives via some edge (u, v) after a walk to u with at most $k-1$ edges:

$$d_k(v) = \min\Big(d_{k-1}(v),\ \min_{(u,v)\in E}\big(d_{k-1}(u) + w(u,v)\big)\Big)$$

Computing round k from round $k-1$ relaxes every edge once: $O(|E|)$ per round. No vertex order needed — *any*

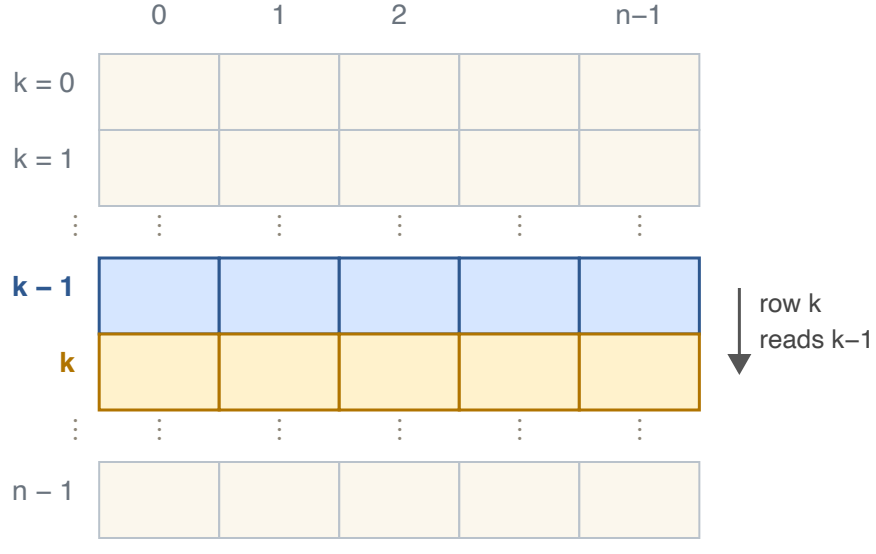
HOW MANY ROUNDS?

Suppose no *negative-weight cycle* is reachable from s. Excising a cycle from a walk changes its weight by the cycle's weight, which is ≥ 0 — so removing cycles never hurts.

Hence some shortest walk repeats no vertex: it is a *simple path*, and a simple path has at most $n - 1$ edges (where $n = |V|$).

> **CONSEQUENCE** If no negative cycle is reachable from s, then $d_{n-1}(v) = d(s, v)$ for every vertex v: $n - 1$ rounds of relaxing all edges suffice.

FROM TABLE TO ROLLING ARRAY



The DP table: one row per round, one column per vertex.

Round k only needs the information from round $k - 1$, so we only need to keep two rows at a time.

UPDATING IN PLACE

Even better: we can update *one* array in place.

Mid-round, an entry may already hold a round-k value when we read it — that only means improvements propagate *sooner*.

After round k we still have $\texttt{dist}[v] \leq d_k(v)$, and every value is the weight of a genuine walk, so after $n - 1$ rounds the answer is exact.

BELLMAN–FORD: CODE

```
struct Edge { int from; int to; double weight; };

std::vector<double> bellmanFord(int n, const std::vector<Edge>& edges, int s) {
    // real infinity: INF + weight stays INF, so no relax guard needed
    const double INF = std::numeric_limits<double>::infinity();
    std::vector<double> dist(n, INF);   // dist plays the role of d_k
    dist[s] = 0.0;
    for (int k = 1; k < n; ++k) {              // rounds k = 1, ..., n-1
        for (const auto& e : edges) {
            if (dist[e.from] + e.weight < dist[e.to]) {
                dist[e.to] = dist[e.from] + e.weight;   // relax
            }
        }
    }
    return dist;
}
```

RUNNING TIME $n-1$ rounds, each relaxing all $|E|$ edges: $O(|V| \cdot |E|)$, with $O(|V|)$ space.

DETECTING NEGATIVE CYCLES

What if there *is* a negative cycle reachable from s? Then walks can get ever cheaper by looping.

Run one extra round: if some $\texttt{dist}[v]$ still improves in round n, a walk with n edges beats every walk with at most $n - 1$ edges. Such a walk must repeat a vertex, and the enclosed cycle must have negative weight.

NEGATIVE-CYCLE TEST Some `dist` value improves in round n $\iff$ a negative cycle is reachable from s.
No improvement means the values already converged to true distances.

ONE FAMILY OF ALGORITHMS

Setting	Algorithm	Time
DAG, any weights	relax in topological order	$O(\|V\| + \|E\|)$
Non-negative weights	Dijkstra (last semester)	$O((\|V\| + \|E\|) \log \|V\|)$
Negative weights, cycles	Bellman–Ford	$O(\|V\| \cdot \|E\|)$

All three are edge relaxation; they differ only in the *order* and *number of times* edges are relaxed — and the DP view tells us why each schedule is

enough.

> **ANIMATION** Step through the rounds in the Bellman–Ford animation — you may remember it as the "Advanced Algorithms preview" at the end of the DSA course. Watch d_k spread one edge further per round, and the round-n check catch a negative cycle.

MAXIMUM SUBARRAY SUM

MAX SUBARRAY SUM

We are given an array **`arr`** of integers:

We want to find indices $i \leq j$ that maximize the sum

$$\mathbf{arr}[i] + \mathbf{arr}[i+1] + \cdots + \mathbf{arr}[j]$$

That is, we want to maximize the sum of entries in a subarray.

EXAMPLE

In this case the maximum sum of a subarray is `6`, realized by the subarray from index 3 to index 5.

BRUTE FORCE

Try all indices $i \leq j$ and compute the sum $\texttt{arr}[i] + \cdots + \texttt{arr}[j]$.

```
int maxSubarraySum(const std::vector<int>& vec) {
    int n = static_cast<int>(vec.size());
    int maxSum = vec.at(0);
    for (int i = 0; i < n; ++i) {
        for (int j = i; j < n; ++j) {
            int currentSum = 0;
            for (int k = i; k <= j; ++k) {
                currentSum += vec.at(k);
            }
            maxSum = std::max(maxSum, currentSum);
        }
    }
    return maxSum;
}
```

What is the running time? Three nested loops: $O(n^3)$.

WASTED WORK

Obviously there is wasted work in computing

$$\mathtt{arr}[3] + \mathtt{arr}[4] + \mathtt{arr}[5] + \mathtt{arr}[6]$$

from scratch when we already have computed

$$\mathtt{arr}[3] + \mathtt{arr}[4] + \mathtt{arr}[5]$$

We can compute all sums starting from index i building on the previous sum.

RE-USING SUMS

```
int maxSubarraySum(const std::vector<int>& vec) {
    int n = static_cast<int>(vec.size());
    int maxSum = vec.at(0);
    for (int i = 0; i < n; ++i) {
        int sumFromi = 0;
        for (int j = i; j < n; ++j) {
            sumFromi += vec.at(j);
            maxSum = std::max(maxSum, sumFromi);
        }
    }
    return maxSum;
}
```

What is the running time of this solution?

Two nested loops: $O(n^2)$. Can we do better?

DYNAMIC PROGRAMMING

Devise subproblems that build up to the full solution.

In array problems, subproblems often focus on prefixes:

$$\texttt{arr}[0], \texttt{arr}[1], \ldots, \texttt{arr}[i]$$

Recipe Compute some value $\texttt{dp}[i]$ of this prefix where:

FIRST ATTEMPT

First attempt to define subproblems:

> What is the maximum subarray sum in $\texttt{arr}[0], \texttt{arr}[1], \ldots, \texttt{arr}[i]$?

1) Computing $\texttt{dp}[n-1]$ solves the problem. ✓

2) Not obvious how to compute $\texttt{dp}[i+1]$ from $\texttt{dp}[0], \ldots, \texttt{dp}[i]$. ✗

SECOND ATTEMPT

Let $\mathtt{dp}[i]$ be the maximum subarray sum that ends at i.

1) The final answer will be the maximum of $\mathtt{dp}[0], \ldots, \mathtt{dp}[n-1]$.

2) Going from $\mathtt{dp}[i]$ to $\mathtt{dp}[i+1]$: the maximum sum ending at $i+1$ either includes i or not.

   If not, then $\mathtt{dp}[i+1] = \mathtt{arr}[i+1]$.

   If yes, the best we can do up to i is $\mathtt{dp}[i]$, so $\mathtt{dp}[i+1] = \mathtt{dp}[i] + \mathtt{arr}[i+1]$.

SECOND ATTEMPT: CODE

$$\mathbf{dp}[i+1] = \max(\mathbf{arr}[i+1],\ \mathbf{dp}[i] + \mathbf{arr}[i+1])$$

```cpp
int maxSubarraySum(const std::vector<int>& vec) {
    int n = static_cast<int>(vec.size());
    int maxSum = vec.at(0);
    int currentSum = vec.at(0);   // currentSum plays the role of dp[i]
    for (int i = 1; i < n; ++i) {
        currentSum = std::max(currentSum + vec.at(i), vec.at(i));
        maxSum = std::max(currentSum, maxSum);
    }
    return maxSum;
}
```

Note: as we only use $\mathbf{dp}[i]$ to compute $\mathbf{dp}[i+1]$, we don't need to store the whole $\mathbf{dp}$ array.

RUNNING TIME

We now have a solution that runs in time $\Theta(n)$.

> **KADANE'S ALGORITHM** This is known as Kadane's algorithm. Its Wikipedia page recounts the interesting history of the problem — worth a read.

The key move: change the subproblem from "best in a prefix" to "best *ending at* i." We will use this trick again.

LONGEST INCREASING RUN: SUBARRAY AND SUBSEQUENCE

THE PROBLEM (SUBARRAY)

We are given an array `arr` of integers:

We want to find the longest subarray containing consecutive (increasing) numbers. We call consecutive increasing integers a run.

In this case the answer is 3, because of the run $7, 8, 9$ from index 3 to 5.

Note that the subarray is contiguous — we cannot include the 6 in index 0.

DP FOR THE SUBARRAY VERSION

We again consider subproblems defined on prefixes $\texttt{arr}[0], \ldots, \texttt{arr}[i]$.

Let's try the subproblem: $\texttt{dp}[i]$ is the longest run ending at index i.

Then the answer to the whole problem is the maximum of $\texttt{dp}[0], \ldots, \texttt{dp}[n-1]$, and the update is

$$\texttt{dp}[i+1] = \begin{cases} 1 & \text{if } \texttt{arr}[i+1] \neq \texttt{arr}[i]+1 \\ \texttt{dp}[i]+1 & \text{otherwise} \end{cases}$$

SUBARRAY VERSION: CODE

```cpp
int longestRunSubarray(const std::vector<int>& vec) {
    if (vec.empty()) {
        return 0;
    }
    int n = static_cast<int>(vec.size());
    int maxRun = 1;
    int currentRun = 1;
    for (int i = 1; i < n; ++i) {
        currentRun = (vec.at(i) == vec.at(i - 1) + 1) ? currentRun + 1 : 1;
        maxRun = std::max(currentRun, maxRun);
    }
    return maxRun;
}
```

Same shape as Kadane: one pass, $\Theta(n)$, constant extra space.

SUBSEQUENCE VARIATION

Now we want the longest subsequence of consecutive integers.

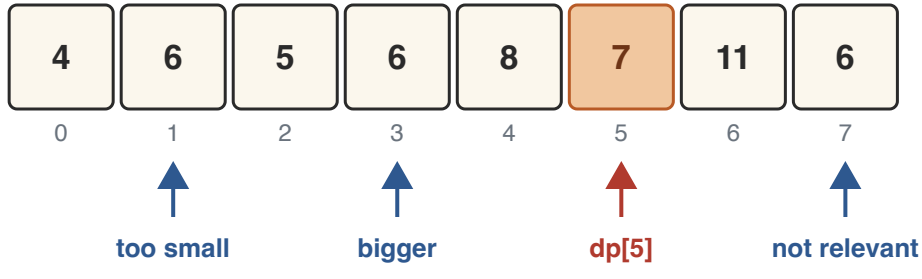
> **DEFINITION** A subsequence of an array is obtained by deleting some entries of the array. Elements appear in order, but do not have to be adjacent as in a subarray.

The longest subsequence which is a run is $6, 7, 8, 9$.

Note that $4, 5, 6, 7, 8, 9$ does *not* form a subsequence — the 6 comes before the 4 and 5.

DP FOR THE SUBSEQUENCE VERSION

Let $\mathtt{dp}[i]$ be the length of a longest run subsequence ending at $\mathtt{arr}[i]$. Because we care about runs, we need the indices j where $\mathtt{arr}[j] + 1 == \mathtt{arr}[i+1]$.



To compute dp[5] we will use dp[j] where arr[j] = 6.

The length of a longest subsequence run ending at 6 cannot decrease when 6 appears later in the array — so only the *latest previous* occurrence matters. A 6 at a position after 5 is not relevant.

THE UPDATE

To compute $\mathtt{dp}[i+1]$ we find the previous occurrence of $\mathtt{arr}[i+1]-1$.

If $\mathtt{arr}[i+1]-1$ has not appeared before, then $\mathtt{dp}[i+1]=1$.

Otherwise, if the previous occurrence is at position j, then

$$\mathtt{dp}[i+1]=1+\mathtt{dp}[j]$$

We need a data structure to look up the previous occurrence of $\mathtt{arr}[i+1]-1$. A dictionary data structure is good for this.

A DICTIONARY AS THE DP TABLE

With a `map` or `unordered_map`, the lookup structure doubles as the DP table: **`runTo`**$[j]$ = longest run up to value j so far.

```
int longestRunSubsequence(const std::vector<int>& vec) {
    int maxRun = 0;
    std::unordered_map<int,int> runTo;
    for (int x : vec) {
        if (runTo.contains(x - 1)) {
            runTo[x] = runTo[x - 1] + 1;
        } else {
            runTo[x] = 1;
        }
        maxRun = std::max(runTo[x], maxRun);
    }
    return maxRun;
}
```

Running time: $O(n \log n)$ with `map`, average-case $O(n)$ with `unordered_map`.

LONGEST INCREASING SUBSEQUENCE

THE PROBLEM

Input: we are given an array of integers.

The array z; the subsequence 4, 8, 9, 11 is highlighted.

Definition An increasing subsequence is a subsequence where each number is larger than all numbers preceding it in the subsequence.

For example: $4, 8, 9, 11$ is an increasing subsequence.

THE GOAL

The goal in LIS is to find the *length* of a longest increasing subsequence.

In this example, the answer is 6. An example realising this is $0, 1, 2, 7, 9, 11$.

DEFINITION OF SUBPROBLEM

This problem sounds like a good candidate for 1D dynamic programming.

Subproblems: what is the longest increasing subsequence ending at $z[i]$? Let this value be $\mathbf{dp}[i]$.

$$\mathbf{dp}[i] = 1 + \max_{j<i,\ z[j]<z[i]} \mathbf{dp}[j]$$

How can we compute this?

THE $O(n^2)$ DP

For each i, iterate over all $j < i$: $\mathbf{dp}[i] = 1 + \max_{j<i,\, z[j]<z[i]} \mathbf{dp}[j]$.

```cpp
int lis(const std::vector<int>& z) {
    int n = static_cast<int>(z.size());
    std::vector<int> dp(n, 1);   // each element alone is increasing
    int best = (n > 0) ? 1 : 0;
    for (int i = 1; i < n; ++i) {
        for (int j = 0; j < i; ++j) {
            if (z.at(j) < z.at(i)) {
                dp.at(i) = std::max(dp.at(i), dp.at(j) + 1);
            }
        }
        best = std::max(best, dp.at(i));
    }
    return best;
}
```

RUNNING TIME $O(n^2)$ — the version we can easily

rediscover.

FASTER THAN $O(n^2)$?

> **STRETCH** Everything to the end of this section is stretch material — a taste of how data structures can accelerate a DP.

There is actually a data structure we can use to solve this problem faster.

Query: what is the maximum of $\mathbf{dp}[j]$ over all j with $z[j] < z[i]$?

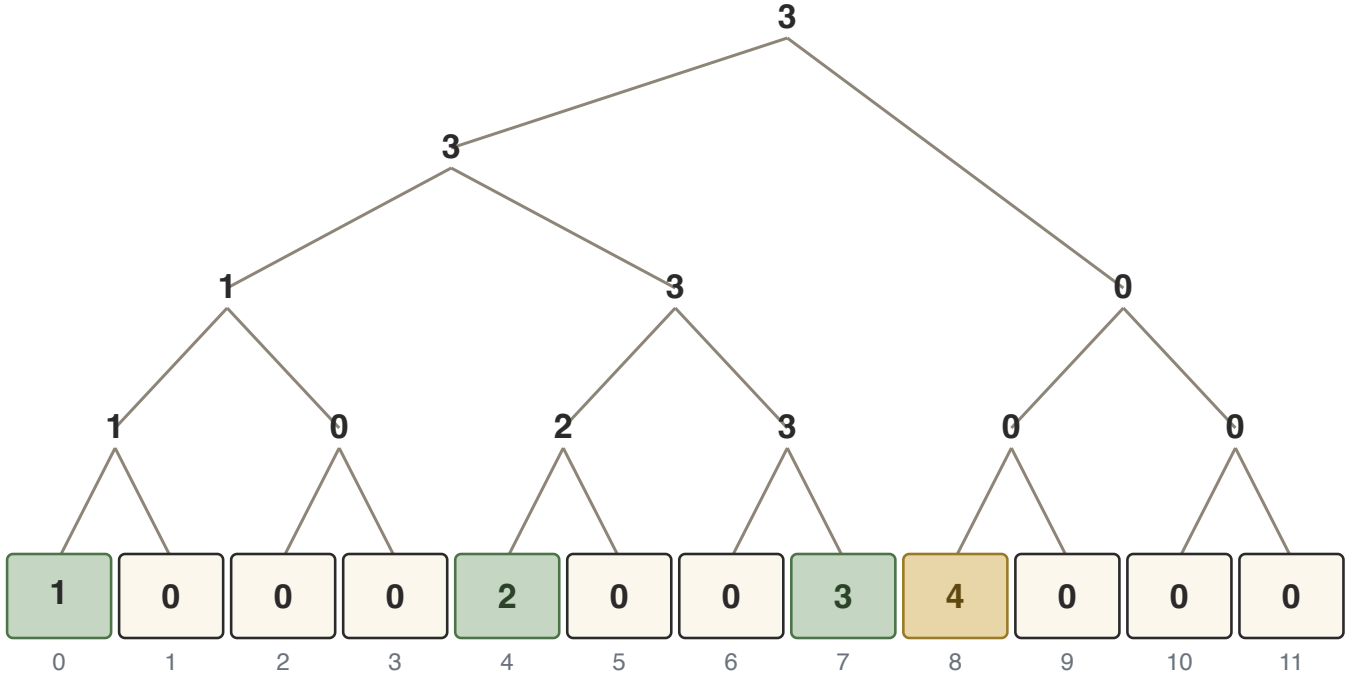
COORDINATE COMPRESSION

Problem: the values $z[j]$ can be any integers.

Solution: only the relative order matters — sort in $O(n \log n)$ and map the values to $0, \ldots, n-1$.

Then a value doubles as an *index* into an array $\mathtt{dp}'$, where $\mathtt{dp}'[v]$ is the length of a longest increasing subsequence ending with value v.

SEGMENT TREE



dp′ after processing 0, 4, 7 — leaves are dp′, each internal node stores the max of its subtree. The update dp′[8] = 4 (gold) refreshes one leaf-to-root path.

The array values form the leaves of a binary tree, where we take the maximum at each node. A prefix-max query $\max_{v<z[i]} \mathbf{dp}'[v]$ checks at

most $O(\log n)$ nodes; an update rewrites the $\log n$ nodes on the path from leaf to root.

Total: $O(n \log n)$ for LIS.

PATIENCE SORTING

STRETCH Another $O(n \log n)$ route — a beautiful algorithm, easier to implement than a segment tree, but not dynamic programming.

Patience sorting is similar to the card game of solitaire. We think of the array entries as "cards" and pass through the array from left to right putting the cards into piles. Each pile is non-increasing: we cannot put a larger card on top of a smaller card.

RULE Place each card on the leftmost pile where it is at most the top card of the pile. If the card is larger than the top card of all piles, start a new pile to the right of all existing piles.

WHY PATIENCE SORTING WORKS

FACT 1 The elements of any increasing subsequence must be in distinct piles: $\#\text{piles} \geq \text{length of a LIS}$.

FACT 2 $\text{length of a LIS} \geq \#\text{piles}$: start with a number in the rightmost pile and follow the arrows back to the first pile — this gives an increasing subsequence with one element per pile.

Fact 1 + Fact 2: $\text{length of a LIS} = \#\text{piles}$. With binary search for the target pile, this runs in $O(n \log n)$.

FUN CONSEQUENCE Any sequence of n numbers has an increasing or a non-increasing subsequence of size $\sqrt{n}$: with fewer than $\sqrt{n}$ piles, some pile has $\geq \sqrt{n}$ cards — and each pile is non-increasing.

COIN CHANGE

COIN CHANGE

In the making change problem we are given a set of coin denominations and a target amount. We have an unlimited supply of each denomination.

We want to find the *minimum number of coins* that sums to the target amount.

For example, with $\texttt{coins} = \{1, 9, 10\}$ and $\texttt{amount} = 18$, the answer is 2 (two 9-coins).

What is the dynamic programming algorithm for this?

SUBPROBLEM AND UPDATE

Subproblems are now *amounts*, not prefixes: let $\mathbf{dp}[a]$ be the minimum number of coins summing to exactly a.

1) $\mathbf{dp}[\mathtt{amount}]$ solves the problem.

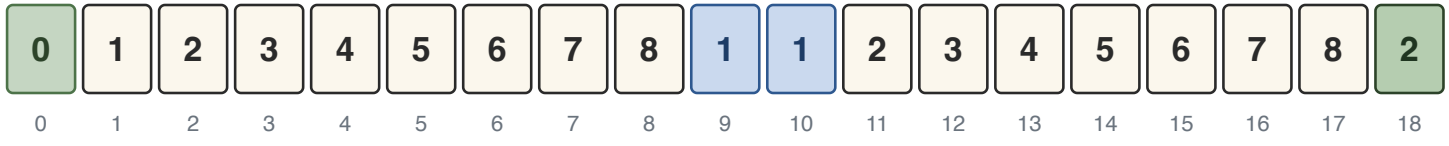
2) The last coin of an optimal solution for a is some denomination $c \leq a$; removing it leaves an optimal solution for $a - c$:

$$\mathbf{dp}[a] = 1 + \min_{c \in \mathtt{coins},\ c \leq a} \mathbf{dp}[a - c]$$

Base case: $\mathbf{dp}[0] = 0$; unreachable amounts stay ∞.

WORKED EXAMPLE

$\texttt{coins} = \{1, 9, 10\}, \texttt{amount} = 18$:



dp[a] for a = 0, …, 18. Base case in green (left); the single-coin amounts 9 and 10 in blue; the answer dp[18] = 2 in green (right).

$$\texttt{dp}[18] = 1 + \min(\texttt{dp}[17],\ \texttt{dp}[9],\ \texttt{dp}[8]) = 1 + \min(8, 1, 8) = 2$$

.

Note how the answer *drops* at $a = 9$: taking the biggest coin first is not always the way — $\texttt{dp}$ considers every last coin.

COIN CHANGE: CODE

```cpp
int coinChange(const std::vector<int>& coins, int amount) {
    const int INF = amount + 1;   // no answer needs more coins
    std::vector<int> dp(amount + 1, INF);
    dp[0] = 0;
    for (int a = 1; a <= amount; ++a) {
        for (int c : coins) {
            if (c <= a) {
                dp[a] = std::min(dp[a], dp[a - c] + 1);
            }
        }
    }
    return (dp[amount] < INF) ? dp[amount] : -1;
}
```

RUNNING TIME $O(\texttt{amount} \times \#\texttt{coins})$.

TEASER Greedy shortcut for coins like $\{1, 5, 25\}$? See Week 4.

WEEK 1 TAKEAWAYS

Problem	Subproblem	Time
Longest path in a DAG	best path *into* v, in topological order	$O(\|V\| + \|E\|)$
Bellman–Ford	shortest walk to v using *at most* k *edges*	$O(\|V\| \cdot \|E\|)$
Max subarray sum	best sum *ending at* i	$\Theta(n)$

Problem	Subproblem	Time
Longest increasing subsequence	longest LIS *ending at* i	$O(n^2)$, stretch $O(n \log n)$
Coin change	fewest coins for *amount* a	$O(\texttt{amount} \cdot \#\texttt{coins})$

WEEK 1 TAKEAWAYS

THE RECIPE Define subproblems whose answers (1) combine to solve the whole problem, and (2) can be computed from earlier subproblems. “Best *ending here*” is often the right refinement.